

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 676 — Data Mining

Finding structure in data too large to look at, without finding structure that is not there

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 10 (with CSCE 638 and CSCE 670)

Prerequisites: CSCE 629 Analysis of Algorithms and CSCE 658 Randomized Algorithms, both used heavily; CSCE 633 Machine Learning for the evaluation discipline; CSCE 678 Distributed Systems for Module 10

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A mining study on a dataset of at least a few million records — a pattern you found, a significance argument that survives the number of hypotheses you tested, a baseline that explains the pattern more cheaply, and a statement of what you would need to call it causal

Course description

Data mining is the search for structure in data too large to inspect — which makes it **two problems rather than one: finding candidate structure efficiently, and deciding whether what you found is real**. **Module 01 establishes both**, and the course alternates between them deliberately rather than treating the second as an afterthought.

The algorithmic half is about trading exactness for feasibility, deliberately and with a stated bound. Hashing, sketching, sampling, and sublinear approximation appear throughout (Modules 02, 06, and 10) — and **they are all applications of CSCE 658's machinery**, which is why that course is a prerequisite rather than a recommendation.

The statistical half is about the fact that searching hard enough guarantees a finding. **Module 11 is the course's centre**: if you test ten thousand hypotheses at the five percent level, you will find five hundred 'significant' patterns in pure noise — **so a pattern found by search needs a different standard of evidence than one predicted in advance**, and most published mining results do not meet it.

The third theme is that scale changes which algorithm is correct. An $O(n^2)$ method is not slow at a billion records; it is impossible (Module 10) — and **the constraint is usually memory and passes over the data rather than arithmetic**, which is why the streaming and sketching material is practical rather than exotic.

And the closing position is about honest claims. **Module 12 covers what mining does to the people in the data**, and **Module 13 is about stating what a found pattern establishes** — because 'we discovered a

'correlation' is a statement about a search procedure, and saying how hard you looked is what makes it interpretable.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain the two problems and why they need separate treatment.
- 2 Find near-duplicates and similar items at scale.
- 3 Mine frequent patterns and assess the rules they produce.
- 4 Cluster large data and evaluate the result honestly.
- 5 Reduce dimension and explain what each method preserves.
- 6 Process a stream with bounded memory and a stated error.
- 7 Mine a graph for communities and centrality.
- 8 Detect anomalies and explain the base rate problem.
- 9 Build and evaluate a recommender.
- 10 Explain what scale does to algorithm choice.
- 11 Correct for multiple testing and defend a found pattern.
- 12 Explain re-identification and the privacy mechanisms.
- 13 State what a discovered pattern honestly establishes.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Two Problems, Not One	You found a pattern. Why should anyone believe it?
2	Similar Items at Scale	How do you find similar pairs without comparing all pairs?
3	Frequent Patterns	Which items appear together more than they should?
4	Clustering at Scale	Are there natural groups, and how would you know?
5	Dimensionality Reduction	Does the data lie near a simpler space?
6	Streams and Sketches	What can you compute seeing each item once?
7	Graph Mining	Who is central, and who belongs together?
8	Anomaly Detection	Which records do not belong, and how would you know?

Wk	Module	Core question it answers
9	Recommendation	What should this user see next?
10	Scale and Systems	When is a cluster the right answer, and when is it not?
11	The Pattern You Found	You searched hard and found something. Now what?
12	Privacy	Can you analyse data about people without exposing them?
13	Claiming a Pattern Honestly	You found something. What can you say?

Projects

Project 1 — Similar items at scale (after Module 6)

Build a near-duplicate detector on a few million documents, with the approximation bounds verified.

Requirements

- **Shingling, minhashing, and LSH implemented yourself** (Module 02), not from a library.
- **The Jaccard estimate's error measured against exact computation** on a sample, and compared to the theoretical bound.
- **The LSH parameters chosen deliberately**, with the S-curve plotted and the threshold justified.
- **The candidate pair count and the false positive and negative rates reported** at your chosen parameters.
- **A runtime comparison** against the exact all-pairs computation, extrapolated honestly.
- **And an inspection of fifty detected pairs**, with the failure categories named.

Done means

- **The empirical error matching the theoretical bound — and an explanation if it does not**, which is the part that demonstrates understanding.
- **The S-curve plotted and the parameters derived from it** rather than tuned by trial.
- **Both error rates reported**, because **reporting only recall is the standard omission** and it hides the trade.
- **And the fifty pairs actually inspected**, with categories that came from the data.

Project 2 — A pattern, defended (after Module 12)

Find a pattern in a large dataset and then attack it until you believe it or do not.

Requirements

- **A pattern found by search**, stated precisely, with the effect size and not only the significance.
- **The number of hypotheses you tested — counted, including the ones you abandoned — and the correction applied** (Module 11).
- **Validation on held-out data you had not touched**, with the result reported whichever way it came out.
- **At least three cheaper explanations tested and ruled out**: a confounder, a sampling artefact, and a data-collection effect.
- **A statement of what you would need to call it causal**, and why you do not have it.
- **And the privacy assessment** (Module 12) if the data concerns people.

Done means

- **The hypothesis count honest — which is graded hardest, because it is the number everyone understates** and the whole correction depends on it.

- **The held-out validation reported whichever way it went. A pattern that failed validation is a complete and passing project**, and a project with no failed validation is suspicious.
- **Three alternatives genuinely tested**, not dismissed in prose.
- **And the causal statement specific: naming the experiment or the instrument you would need** is the deliverable, not a disclaimer.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Leskovec, Rajaraman & Ullman — Mining of Massive Datasets (free PDF)	The primary text, free in full from the authors. Modules 02, 04, 06, 07, and 09 follow it closely, and its treatment of locality-sensitive hashing is the best available.
Tan, Steinbach, Karpadne & Kumar — Introduction to Data Mining, 2nd edition	The broader companion, with free chapters. Stronger on Modules 03, 05, and 08, and on the evaluation material generally.
Aggarwal — Outlier Analysis, 2nd edition	Module 08 in depth — and honest about how badly anomaly detection is usually evaluated. Library copy.
Benjamini & Hochberg, and the replication literature (free)	Module 11's methods and its motivation. The FDR paper itself, read alongside the reproducibility work that made it urgent.
Dwork & Roth — The Algorithmic Foundations of Differential Privacy (free PDF)	Module 12's formal half, free in full. Read the first three chapters; the definition is what matters for this course.
Stanford CS246 — Mining Massive Data Sets (free)	The course this one is shaped against , with free lectures and assignments by the authors of the primary text.

Tooling

- **Python with NumPy, SciPy, scikit-learn, and pandas** — and **the exercises require you to implement the core algorithms before using the library versions**, because the approximation guarantees are invisible from outside.
- **A dataset of at least a few million records. Below that, everything fits in memory and the course's central constraint disappears** — which makes the material feel arbitrary. Public options are listed in the map.
- **Spark or Dask** for Module 10 — **run it on one machine first**, which is where you learn what the partitioning is doing, and only then on more.
- **datasketch and networkx** for Modules 06 and 07. **Read datasketch's error parameters and verify its bounds empirically**, which is Module 06 §4's exercise.
- **A notebook, and a discipline about it: record every hypothesis you test, because Module 11 requires the count** and reconstructing it afterwards is not possible.
- **And a held-out portion you look at once. The same requirement as CSCE 638's and CSCE 633's**, and in this course it is the only defence against Module 11's problem.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.